Wayne Williams Opanja

Nairobi, Kenya
+254 798 609 919 · waynewilliams2028@gmail.com · wayne-spec.github.io

Project Portfolio

Software engineer and product manager. I build and ship data platforms, AI products, fintech and digital public services across Africa, from the first conversation with users to a system running in production. On my most recent engineering team I finished first for delivery, with 297 issues completed at a 90.5% completion rate. Walkthroughs and demos are available on request.

At a glance

Project	Client / context · where	Role	Dates	Result
National Innovation Portal (RICC)	Federal Ministry of Education Nigeria	Lead Frontend Engineer	Jan 2026 – Jun 2026	Live national platform, survey to funding
NGAO	Last-mile reporting Kenya	Full-Stack Developer	Mar 2026 – Jun 2026 (development); rolling out	Rolled out; 3,000+ users
Kenya Climate & Nature Directory	Climate Shapers Africa Africa	Sole Engineer	Jul 2026 – present	Running nightly; 2,774 records migrated
PropFlow	Landlords and managing agents Kenya, Uganda and Rwanda	Lead Engineer and Product Lead	Jun 2026 – Sep 2026 (development); live and maintained	In production; users in Kenya, Uganda and Rwanda
Cross-Border Investment Platform	Fintech, diaspora investment US and Europe to Kenya, Nigeria, Mauritius and South Africa	Lead Frontend Engineer, with backend, QA and research	Aug 2025 – Jun 2026	Live in production; four portals
Revenue Intelligence Assistant	Nairobi City County Kenya	Lead Engineer	Mar 2026	Revenue answers in seconds, audited
Breach Alert	Individuals and companies Global	Sole Engineer	Sep 2026	Live; dark-web watcher plus 3 breach sources
Harambee Shop	Households and students Kenya	Sole Engineer and Product Owner	Sep 2026 – present	Deployed; live pilot with real payments
Course Evaluation Platform	University teaching quality Strathmore University, Kenya	Sole Engineer	Jul 2026 – present (first version Feb 2025)	In use by academic directors
Climate-Stress Analytics for Banks	Kenya Bankers Association, Africa Climate & Nature Data Platform Kenya and Tanzania	Technical Analyst and Developer	Sep 2025 – Nov 2025	Climate risk turned into default risk

MessageHeard	Media production United States and United Kingdom	Full-Stack Developer	Nov 2025 – Jan 2026	Multi-team platform, transcript to video
Stock Research Assistant	Nairobi Securities Exchange analysts Nairobi, Kenya	Lead Engineer	May 2026 – present	In use; sold to another business
Tumaini Support & Care	Healthcare staffing company United States	Sole Engineer	Aug 2026	Live; candidate, employer and admin portals
Avodah	Poultry farmers (university capstone) Kenya	Developer	Nov 2024 – 2026	94% top-3 retrieval in English and Kiswahili
Climate Risk Management Toolkit (prototype)	British International Investment Global	Sole Engineer	Sep 2026 – Oct 2026	Working prototype, in proposal and review
Trademark Registry Upgrade	Federal Ministry of Industry, Trade and Investment Nigeria	Software Engineer	Sep 2025 – May 2026	Upgraded registration system in use
Chat Your Finances	Personal finance Kenya	Lead Developer	Jan 2026 – Jun 2026	AI assistant on WhatsApp, Telegram, web

Projects

1. National Innovation Portal (RICC)

Client: Federal Ministry of Education · **Where:** Nigeria · **Role:** Lead Frontend Engineer · **Dates:** Jan 2026 – Jun 2026

Problem and goal. Nigeria had no reliable view of how research moves from universities, institutes and innovators into the market. Data came in through Google Forms, often from areas with weak or no internet, duplicate reports were common, and funders had no way to find investable work. The goal was one national platform that collects trustworthy data from every stakeholder group, shows where innovations stall, and connects the best of them to funding.

Key contributions

- **Research:** found why universities, industry, investors and regulators weren't connecting, and turned that into user journeys and stakeholder questionnaires.
- **Frontend lead:** main frontend engineer across almost every module, alongside the backend lead, with 93 merged pull requests between January and June 2026.
- **Offline-first data collection:** field agents complete long questionnaires with no connection, and answers sync automatically once they're back online. The questionnaires hide questions that don't apply and include repeating groups, matrices, file uploads, word limits, ranked choices, location logic, consent tracking and reference numbers.
- **Sign-off and verification:** e-signing by the respondent and an authorising officer, send-back for corrections, an evidence-verification panel for the secretariat, and signed PDF receipts.
- **Analytics:**
  - a Nigeria heat map and a "valley of death" chart showing where innovations drop off
  - readiness scorecards, funding maps, sector and return-on-investment charts
  - a qualitative tagging dashboard and a chatbot that answers questions about the data with live charts
  - Excel exports, after replacing a vulnerable spreadsheet library
- **Innovators and IP:** registration and profiles, a claim flow for existing records, IP readiness with document upload, an inventorship register with co-inventor consent, trade-secret access behind an NDA, and technology-readiness self-assessments with eligibility results.
- **Matchmaking and funding:**
  - a "For You" feed matching innovators to funders, and an innovation feed for funders (save, dismiss, express interest)
  - funder team management and the admin funding-agreement and milestone set-up
  - three-party milestone approval, dispute threads and a funded-innovator dashboard
  - the M&E dashboard and public record pages
- **Testing and security:** led about 30 bug-fix pull requests in the June QA cycle, including security fixes (path traversal, spreadsheet formula injection, personal data in audit logs), and wrote the pre-handover QA pack of 142 test cases across 20 modules.

Result. Live at ricc.education.gov.ng on the government's own servers, and grown from a national survey into a full innovator-to-funding pipeline. Historical data was cleaned so 62 submissions became 298 distinct innovations, with 105 duplicates removed.

Tools: Next.js · React · TypeScript · offline web app (service workers) · SurveyJS · Recharts · Leaflet · FastAPI · PostgreSQL with pgvector

2. NGAO: WhatsApp Data Collection Platform

Context: last-mile reporting · **Where:** Kenya · **Role:** Full-Stack Developer · **Dates:** Mar 2026 – Jun 2026 (development); rolling out

Problem and goal. Field reporting such as tree planting and seedling sales ran on paper and informal messages. It was slow, hard to verify and easy to fake. The goal was to let people report through WhatsApp, with every submission kept as a permanent record administrators can trust.

Key contributions

- **Conversation engine:** built the engines and orchestrators that run each conversation. Flows are button-led, in English and Kiswahili, and accept text, photos, voice notes and location.
- **Answer correction:** misspelled answers are matched against a dictionary first, then AI. The person confirms ("Did you mean…?"), and the engine learns from every confirmation.
- **Photo checks:** each photo's location data is compared with the shared location, and mismatches are flagged without blocking the reporter.
- **Dashboards:** connected the backend to the admin dashboards, and wrote the reporting reference that checks every dashboard figure against the raw records and the CSV export.
- **No-code pipelines:** a schema builder lets administrators launch new pipelines (tree planting, seedling marketplace, purchasing) without new code.

Result. Development finished in June 2026; rolled out and growing, with more than 3,000 users.

Tools: NestJS · PostgreSQL · React · TypeScript · WhatsApp Business API · Twilio (voice, images) · LLMs

3. Kenya Climate & Nature Directory: Research and Publishing Pipeline

Client: Climate Shapers Africa · **Where:** Africa · **Role:** Sole Engineer · **Dates:** Jul 2026 – present

Problem and goal. Keeping the directory current was manual: searching Google for PDFs, judging relevance by eye, typing metadata by hand, reconciling authors in Excel, uploading covers one by one and loading the site ten rows at a time. The goal was to automate everything except the chief editor's final approval, at almost no running cost.

Key contributions

- **Discovery:** a nightly pipeline searches six research sources (World Bank, OpenAlex, Crossref, Semantic Scholar, arXiv, CORE), removes duplicates, downloads and checks each PDF, extracts the text and makes a cover image.
- **AI review:** AI decides whether each document is about Kenya, East Africa or Africa and climate, keeping a reason for every rejection, then extracts the title, year, publisher, authors, document type, abstract and themes, with OCR for scanned pages and automatic switch-over between AI providers.
- **Human approval:** rules in code check every result before it joins the chief editor's queue. The editor approves, edits or rejects with a reason, and approval publishes the record and rebuilds the public site.
- **Reliability:** each document saves its progress, so a crash or a used-up AI quota simply resumes on the next run. Weekly link checks, author enrichment, record repair and database backups run on their own.
- **Migration:** moved 2,774 existing records into a new database with row-level security.
- **Beyond the directory:**
  - News Studio: daily climate stories written from the source text, with AI-checked header images
  - Content Studio: turns papers into LinkedIn posts, carousels and explainer videos
  - an opportunity tracker that emails Climate Shapers a daily digest of relevant grants and tenders
  - everything an AI writes waits for human approval

Result. Running in production every night inside free tiers; the only running cost is the domain.

Tools: Python · SvelteKit · Astro · Supabase (PostgreSQL) · Cloudflare Pages and R2 · GitHub Actions · LLMs (Cerebras, Groq, OpenRouter) · Remotion

4. PropFlow: AI and Automated Property Management

Context: landlords and managing agents · **Where:** Kenya, Uganda and Rwanda · **Role:** Lead Engineer and Product Lead · **Dates:** Jun 2026 – Sep 2026 (development); live and maintained

Problem and goal. Landlords spend their days on routine work: invoices, rent follow-ups, receipts, repairs and the messages around them, spread across spreadsheets, WhatsApp threads and paper files. The goal was a system that does that work automatically, so landlords get reports instead of chasing tasks.

Key contributions

- **Automation:** designed and built it with a partner. The rent roll, invoicing and billing, arrears and late fees, deposits, reconciliation and tenant receipts run automatically; leases are generated and signed online; and maintenance requests flow to contractors with approvals.
- **Reports on WhatsApp:** landlords can ask the system for reports on WhatsApp; tenants report issues there with photos, and emergencies trigger an automatic phone call.
- **Safe AI:** AI proposes the next step, and code checks every suggestion against strict rules before anything happens.
- **Production:** 90 data models, 120 test files, containerised deployment, a background scheduler, nightly backups, error monitoring and a written recovery plan.
- **Expansion:** met partner companies in Uganda and Rwanda, including the leadership teams of Keystone Technologies and Unispace, and trained five CEOs, property owners and their CTOs in Rwanda.

Result. In production with users in Kenya, Uganda and Rwanda. Ongoing maintenance.

Tools: Next.js · TypeScript · Prisma · PostgreSQL · Twilio · WhatsApp · Paystack · Docker · Sentry

5. Cross-Border Investment Platform

Context: fintech, diaspora investment · **Where:** US and Europe to Kenya, Nigeria, Mauritius and South Africa · **Role:** Lead Frontend Engineer, with backend, QA and research · **Dates:** Aug 2025 – Jun 2026

Problem and goal. People living abroad had no simple, trusted way to invest back home from the money-transfer and banking apps they already use. The goal was an investment marketplace that partner apps can embed, with identity checks and compliance built in.

Key contributions

- **Frontend:** main frontend engineer for eleven months. Built the partner, fund-manager and admin portals and sign-in: investor onboarding with identity verification (KYC), the marketplace, wallet funding, KPIs and charts, portfolios, transactions, product creation and review, an approvals queue, team invitations and an investment journey timeline.
- **Hardening:** patched vulnerable dependencies, added a logger that strips personal data, security headers, exact money handling, strict role checks, error boundaries, accessibility checks and test-coverage gates in CI.
- **Backend:** built the partner and fund-manager dashboard APIs, the cash-out flow with investor consent, organisation invitations and stand-in payment rails so the full investment flow could be demonstrated end to end. Found and fixed a production outage on the fund-manager dashboard.
- **Research:** wrote the roadmap audit, the workflow explainer and the frontend-to-backend API specification.
- **QA and UAT:** fixed QA findings, reviewed teammates' pull requests and ran end-to-end QA checklists ahead of user acceptance testing.
- **Constraints:** designed around regulatory and foreign-exchange rules across several countries and a key provider changing mid-build; every amount reconciled exactly, and security was planned in advance with an incident playbook.

Result. Signed off in user acceptance testing, deployed to production, and handed over with source code and documentation.

Tools: Next.js · React · TypeScript · NestJS · Python · PostgreSQL · Docker

6. Revenue Intelligence Assistant

Client: Nairobi City County · **Where:** Kenya · **Role:** Lead Engineer · **Dates:** Mar 2026

Problem and goal. Revenue staff searched monthly spreadsheets in different layouts to answer simple questions about collections, and some money was counted twice. The goal was correct answers in plain words, in seconds.

Key contributions

- **Data import:** built a tested pipeline that reads every report layout into one clean database.
- **AI assistant:** staff ask questions on WhatsApp or the web, and only approved phone numbers get answers.
- **Safety:** the AI has read-only access through a restricted database account, so it can never change the county's records, and it switches AI provider automatically if one fails.
- **Accuracy:** audited answers against the original reports, and fixed double-counting and wrong-period results before staff relied on it.

Result. Staff get revenue figures from the county's own data in seconds.

Tools: Next.js · TypeScript · Supabase/PostgreSQL · LLMs · WhatsApp · Vitest

7. Breach Alert: Dark-Web and Data-Breach Monitoring

Context: individuals and companies · **Where:** Global · **Role:** Sole Engineer · **Dates:** Sep 2026

Problem and goal. People and companies usually find out their passwords or data are in criminal hands too late, and too vaguely, to act on the same day. The goal was alerts precise enough to act on immediately.

Key contributions

- **Proof of ownership:** people verify what they monitor: their sign-in email, extra addresses by a time-limited six-digit code, and company domains by a DNS record.
- **Three data sources:** breach databases, infostealer malware logs and ransomware leak-site listings. Duplicates are removed, each finding gets a severity, and every alert comes with plain-language next steps for people and for companies.
- **Dark-web watcher:** a separate service that reads only approved leak-site pages over Tor, never follows links or downloads files, keeps only masked extracts, and matches against monitored addresses "blind" using one-way fingerprints, so it never holds the watchlist.
- **Privacy by design:** emails are encrypted at rest and matched by keyed fingerprints, so a copy of the database reveals nothing; passwords are checked by sending only five characters of their hash and are never stored.
- **Operations:** urgent alerts go out immediately and minor ones in a weekly summary; an admin review desk holds uncertain matches until a person approves them; 114 automated tests; three services deployed with health checks.

Result. Went live covering 169 ransomware leak sites; its first test account surfaced one critical and two high-severity findings.

Tools: Next.js · TypeScript · Drizzle · PostgreSQL · Clerk · Resend · Tor · Docker · Render

8. Harambee Shop: Group Buying at Wholesale Prices

Context: households and students · **Where:** Kenya · **Role:** Sole Engineer and Product Owner · **Dates:** Sep 2026 – present

Problem and goal. In Kenya, wholesale prices run 13–33% below retail, but only in bulk that a single household can't afford or store. The goal was to let neighbours buy wholesale together, safely, with M-Pesa.

Key contributions

- **Shared baskets:** a group fills whole supplier packs together, each person takes single items and pays their share by M-Pesa, and the goods are bought wholesale and delivered on Saturday.
- **Pricing engine:** passes half of every saving to the customer, with a discount ladder that rewards taking more of a pack, and an audit log on every money and pricing decision.
- **Payments:** verified M-Pesa confirmations, account credit, re-pricing when a basket closes, refunds, and cash-outs to M-Pesa with automatic retries.
- **Pilot operations:** imported and checked a real supplier catalogue against retail prices, set owner-approved risk limits, and built the Saturday purchase planner, receiving checks and delivery tracking.
- **Testing:** test programs for the backend, about 40 API test suites, failure-forcing mocks for payments and messaging, and browser tests with zero accessibility findings on mobile and desktop.

Result. Deployed and running a controlled pilot around Strathmore and Madaraka, with real M-Pesa payments.

Tools: Free Pascal (mORMot2) · Next.js · PostgreSQL · Redis · Clerk · Paystack · WhatsApp · Docker · Render

9. Course Evaluation Platform

Context: university teaching quality · **Where:** Strathmore University, Kenya · **Role:** Sole Engineer · **Dates:** Jul 2026 – present (first version Feb 2025)

Problem and goal. Course feedback was low and hard to act on: students worried about anonymity, and directors couldn't see which courses needed attention. The goal was honest, anonymous feedback that leads to action.

Key contributions

- **Anonymous by design:** students need no account, just a short random link; no identity or IP address is stored, while duplicate submissions are still blocked.
- **Director dashboards:** alarm and applause thresholds per department, trends, and round-to-round comparisons that flag a change of lecturer and skip unfair comparisons.
- **AI summaries:** comments sorted into "going well" and "needs attention" with sentiment, released only once enough students have responded, and correctable by reviewers.
- **Follow-through:** follow-up tasks with owners, due dates and evidence; email reports, commendations and alarms; CSV, Excel and branded PDF exports limited to what each role may see.
- **Security and testing:** hashed passwords, a permission check and audit entry on every change, data kept separate per institution, and 49 automated tests in CI.

Result. In use by the academic directors to run evaluations and act on the results. It rebuilds an earlier Django and React version that increased student feedback rates and gave staff timely analytics.

Tools: Next.js · React · TypeScript · Prisma · PostgreSQL · Vercel · Claude / NVIDIA NIM

10. Climate-Stress Analytics for Banks

Client: Kenya Bankers Association, Africa Climate & Nature Data Platform · **Where:** Kenya and Tanzania · **Role:** Technical Analyst and Developer · **Dates:** Sep 2025 – Nov 2025

Problem and goal. Climate risk wasn't part of how banks judged credit risk, least of all for small businesses, whose exposure to drought, floods or a carbon price depends on where they are and what they do. The goal was to show lenders that link in numbers.

Key contributions

- **County risk map:** a small-business climate-risk dashboard covering all 47 Kenyan counties, with inputs for county, industry, business model, asset type and asset value.
- **Risk scoring:** physical risk (drought, flood, heatwave, storm) and transition risk (sector carbon intensity, exposure to carbon pricing).

- **Credit impact:** turns the climate score into probability of default, loss given default and expected credit loss.
- **Context for lenders:** a scenarios guide (baseline, optimistic, pessimistic) and banking-sector views of exposure by sector, green lending and carbon intensity, with NCBA, Equity Bank and Victoria Commercial Bank among the stakeholders.

Result. A working analytics prototype that shows how climate risk changes a small business's chance of default and expected loss, county by county.

Tools: Next.js · React · TypeScript · Recharts · Leaflet · Vercel

11. MessageHeard: AI Video Production Platform

Context: media production · **Where:** United States and United Kingdom · **Role:** Full-Stack Developer · **Dates:** Nov 2025 – Jan 2026

Problem and goal. A team was producing AI-generated video from podcast transcripts with a single-user prototype that couldn't support several teams at once. The goal was a production platform several organisations can use in parallel.

Key contributions

- **Full stack:** built the backend and the interface, and trained the custom image models.
- **Teams and access:** sign-in, roles, invitations and separation between organisations.
- **Production workflow:** transcripts break into timed scenes, with an asset library of characters and objects and AI prompt suggestions for each scene.
- **Generation queue:** image and video jobs run in parallel with live status, retry and cancel.
- **Delivery:** worked to an 8.5-week plan with weekly client check-ins, CI/CD and a staging environment for client testing.

Result. A working multi-team platform that turns a transcript into finished scene images and video.

Tools: Next.js · TypeScript · Python · PostgreSQL · Clerk · image and video models · LoRA

12. Stock Research Assistant

Context: Nairobi Securities Exchange analysts · **Where:** Nairobi, Kenya · **Role:** Lead Engineer · **Dates:** May 2026 – present

Problem and goal. Analysts spend hours gathering prices, Central Bank data and news before analysing a stock. The goal was to automate the first round of research.

Key contributions

- **Multi-agent pipeline:** adapted an open-source multi-agent framework into TypeScript: analyst agents study prices, the economy and news, debate the case for and against, a trader agent makes the call and a risk check tests it against Kenyan inflation.
- **Daily run:** runs automatically every morning before the market opens, with automatic AI-provider failover and a searchable research knowledge base.
- **Platform:** a stock screener, company fundamentals pages and daily brief videos.

Result. Used by analysts in Kenya for research and decision support, and later sold to another business, where it is still running.

Tools: Next.js · TypeScript · Supabase · Vercel AI SDK · GitHub Actions · Redis

13. Tumaini Support & Care: Healthcare Staffing Platform

Client: healthcare staffing company · **Where:** United States · **Role:** Sole Engineer · **Dates:** Aug 2026

Problem and goal. The client needed one place to match nurses and caregivers with care employers and to handle urgent staffing requests, instead of email and phone calls.

Key contributions

- **Candidate portal:** profiles with skills, availability and certifications with expiry dates, private CV upload, job applications and status tracking from submitted to offered.
- **Employer portal:** post jobs for review, review applicants, invite team members and raise staffing requests marked standard, urgent or emergency.
- **Back office:** moderation and verification of employers, a staffing-request pipeline from new to fulfilled, site content, and an audit log of every admin action.
- **Security and testing:** every page and action re-checks the user's role on the server; rate limiting, email notifications, and browser and unit tests on the main flows.

Result. Live, with separate portals for care workers, employers and the client's staff.

Tools: Next.js · React · TypeScript · Prisma · PostgreSQL (Neon) · Upstash Redis · Resend · Vercel

14. Avodah: Bilingual Farming Assistant

Context: poultry farmers (university capstone) · **Where:** Kenya · **Role:** Developer · **Dates:** Nov 2024 – 2026

Problem and goal. Small poultry farmers need reliable answers in the language they speak, and a chatbot that guesses is worse than none. The goal was an assistant that finds the right answer, or says it doesn't know.

Key contributions

- **Assistant:** a WhatsApp assistant over a curated poultry knowledge base, in English and Kiswahili.
- **Measured retrieval:** compared four ways of finding the right answer (keyword search, multilingual embeddings, a hybrid and an LLM-guided method) and three ways of handling Kiswahili questions, on a 40-question test set.
- **Answer quality:** judged answers were 85–88% faithful to the source, and every out-of-scope question was refused.

Result. The best design found the right answer section in its top three results 94% of the time (88% for Kiswahili) and its first result 79% of the time.

Tools: Next.js · TypeScript · PostgreSQL with pgvector · multilingual embeddings · Groq LLMs · Vitest · Playwright

15. Climate Risk Management Toolkit (prototype)

Client: British International Investment · **Where:** Global · **Role:** Sole Engineer · **Dates:** Sep 2026 – Oct 2026

Problem and goal. BII's climate and nature risk toolkit for financial institutions lives in guidance documents and spreadsheets: users must read the guidance, work through the spreadsheets and assemble the outputs themselves. The goal was one guided web app that does the work with them.

Key contributions

- **Guided start:** asks for the user's role, goal and task and builds a combined work plan.
- **All 23 modules:** working tools for every module, with dedicated journeys for institution maturity, client climate screening and financed emissions, and calculators such as stress testing (expected loss, baseline versus stressed).
- **Linked records:** institutions, clients, exposures and portfolios reuse saved inputs, with CSV import, drafts with revision history and PDF, Excel, Word and JSON exports.
- **Accuracy:** a source audit found errors in the earlier app and in BII's own workbooks, which the prototype corrects; captioned learning videos are built from BII's diagrams.

Result. A working prototype of all 23 modules, received positively and now in the proposal and review stage.

Tools: Next.js · React · TypeScript · Prisma · PostgreSQL · ExcelJS · Render

16. Trademark Registry Upgrade

Client: Federal Ministry of Industry, Trade and Investment · **Where:** Nigeria · **Role:** Software Engineer · **Dates:** Sep 2025 – May 2026

Problem and goal. The ministry's existing trademark system needed an upgrade to keep up with registrars and applicants. The goal was faster, more reliable registration and administration.

Key contributions

- **Workflows:** rebuilt the registration and administration workflows on the upgraded system.
- **Data integrity:** kept records consistent from filing to approval.
- **Usability:** made the admin screens easy for non-technical staff.

Result. An upgraded registration and administration workflow in use by the ministry's administrators.

17. Chat Your Finances: AI Finance Assistant

Context: personal finance · **Where:** Kenya · **Role:** Lead Developer · **Dates:** Jan 2026 – Jun 2026

Problem and goal. People struggle to see where their money goes and find filing taxes stressful. The goal was an assistant that answers from their own records.

Key contributions

- **Statement reading:** reads M-Pesa, bank and payslip statements, including scanned and password-protected PDFs, using OCR.
- **Answers:** answers money questions on WhatsApp, Telegram and the web from the user's own records, using retrieval-based AI.
- **Tax and billing:** a tax engine reads P9 forms to guide KRA filing; subscriptions are billed through Paystack.

Result. An assistant that answers money and tax questions from a person's own statements.

Tools: Next.js · Supabase · OCR · RAG · Paystack

More projects

- **Phone-billing system rebuild (Jan – Feb 2026):** rebuilt a university's phone billing as an Odoo module, synced with call records and HR/ERP data. Python · Odoo · Asterisk
- **Penetration test: Harambee SACCO (2024):** tested a financial cooperative's systems from an attacker's point of view and set out the fixes.
- **Event-streaming pipeline (Apr – Jul 2026):** Apache Kafka on a three-broker cluster, Python producer and consumer services in Docker, and PostgreSQL.
- **Cattle disease image classifier (Feb – Mar 2026):** developer on a team project that identifies foot-and-mouth disease, lumpy skin disease and mastitis from photos of cattle. Transfer learning (EfficientNetB0) on 3,921 images reached 91% validation accuracy (macro F1 0.90), served through a Streamlit app in Docker. TensorFlow · Keras · scikit-learn · Streamlit
- **KCSE 2025 results map (Jan 2025):** a county-by-county map of Kenya's top KCSE grades, built from the published results. Python · folium